Task 1: Stride Pattern Prefetcher (SPP)

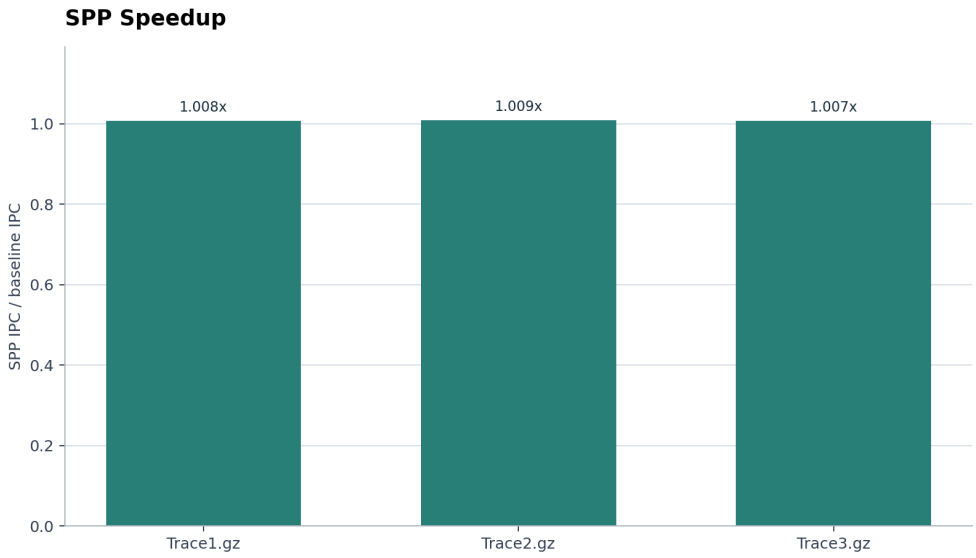
1) IPC, Speedup, L1D Load MPKI, Prefetch Accuracy, and Coverage

The table below reports the values obtained for the three traces.

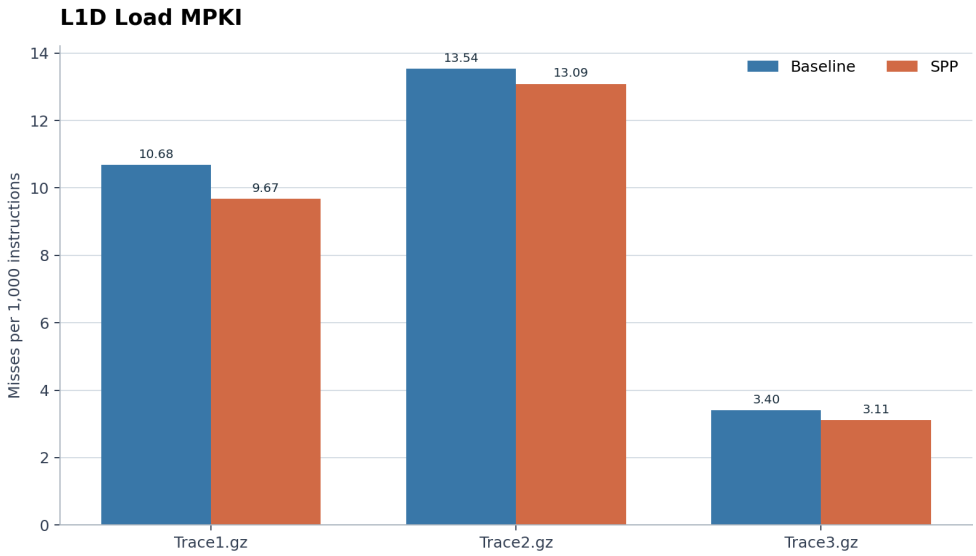
Trace	IPC with Prefetcher	Speedup	L1D Load MPKI	Prefetch Accuracy	Coverage
Trace1.gz	0.233918	1.00778x	9.67160	62.0792%	9.43%
Trace2.gz	0.252293	1.00924x	13.08920	56.0933%	3.31%
Trace3.gz	0.384546	1.00701x	3.10912	34.1697%	8.65%

The SPP metrics were collected by rerunning the prefetcher on all three traces.

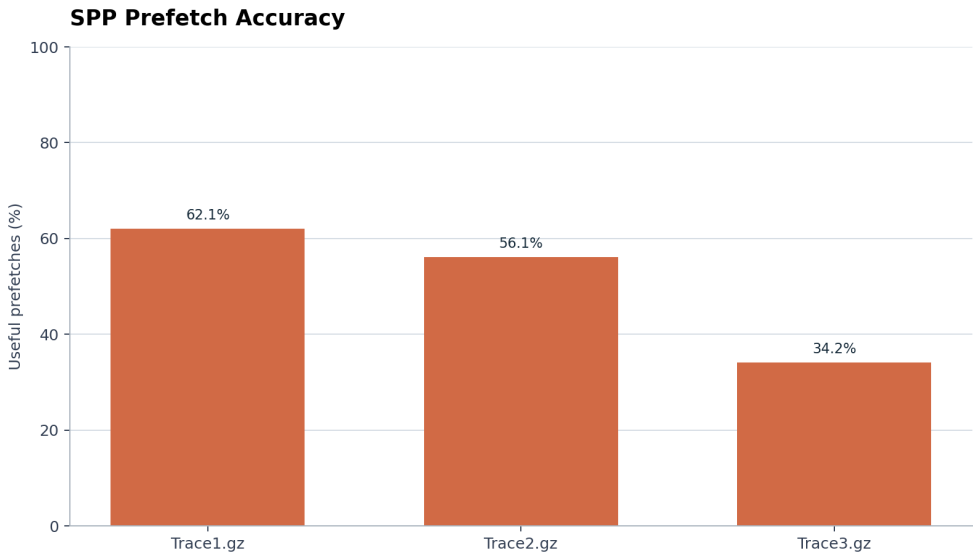
Speedup



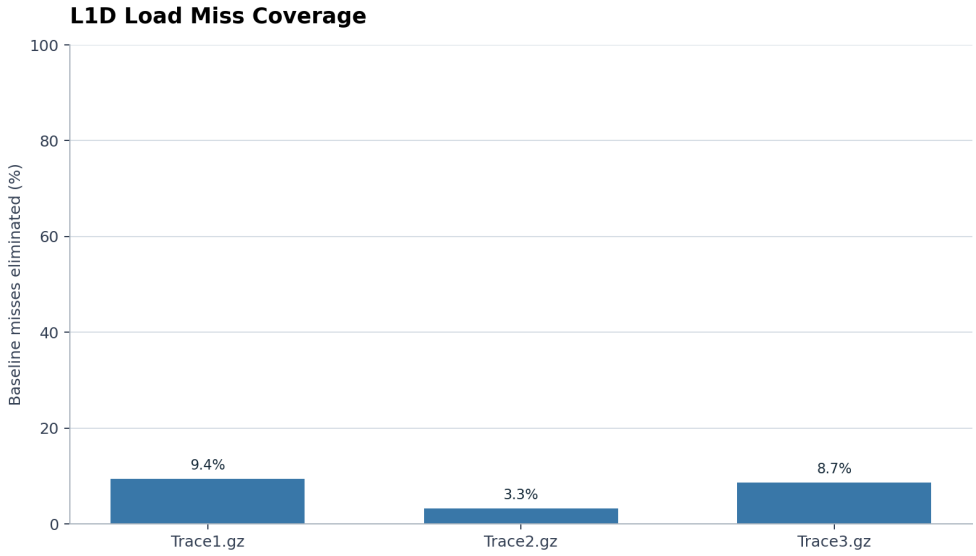
L1D load MPKI



Prefetch accuracy



Coverage



2) Storage overhead of the prefetcher

The prefetcher uses two main tables:

IP tracker table

Each entry of `IP_Tracker` contains:

- `ip` = 64 bits
- `last_addr` = 64 bits
- `last_stride` = 64 bits
- `uniform_conf` = 32 bits

- `signature` = 16 bits
- `lru_rank` = 8 bits

Logical field width:

- 64 + 64 + 64 + 32 + 16 + 8 = 248 bits

There are 128 entries:

- 128 × 248 = 31,744 bits

Pattern table

Each entry of `Pattern_Entry` contains:

- `signature` = 16 bits
- `next_stride` = 64 bits
- `lru_rank` = 8 bits

Logical field width:

- 16 + 64 + 8 = 88 bits

There are 256 entries:

- 256 × 88 = 22,528 bits

LRU uses stack ranks: rank 0 is most recently used, and the largest rank is selected for eviction. Each rank field is `uint8_t`; the 256-entry pattern table uses ranks 0 through 255.

Total storage overhead

- IP tracker table: 31,744 logical bits
- Pattern table: 22,528 logical bits
- Prefetch tracking array: 64 × 64 = 4,096 bits

Total logical field storage per core:

- 31,744 + 22,528 + 4,096 = 58,368 bits (7,296 bytes)

These logical bit counts exclude C++ struct padding. With the current field order and typical 8-byte alignment, `sizeof(IP_Tracker)` is 32 bytes and `sizeof(Pattern_Entry)` is 24 bytes; including the 4,096-byte IP table, 6,144-byte pattern table, and 512-byte prefetch tracking array, the allocated arrays occupy about 10,752 bytes per core.

3) Threshold for accuracy throttling

We chose the accuracy thresholds to be 0.75 and 0.40. The prefetcher increases its degree only when more than 75% of recent prefetches are useful, and it backs off when usefulness drops below 40%. Keeping a gap between these two points makes the control more stable, so the prefetcher does not keep flipping between aggressive and conservative behavior.

These values were picked after trying different cutoffs and comparing IPC, L1D MPKI, and prefetch usefulness across the traces. The final pair gave the best balance between better coverage and lower cache pollution. It lets the prefetcher stay aggressive when the pattern is working but back off quickly when it starts generating useless fills.

4) Why these prefetch degrees were chosen

The prefetcher starts with a degree of 2 and then adjusts itself by one step at a time based on how useful the recent prefetches actually are.

- If accuracy is above 75%, the degree is increased by 1 to improve coverage.
- If accuracy falls below 40%, the degree is decreased by 1 to avoid useless prefetches.

This starting value was chosen because it is a good middle ground: it is high enough to help when the access pattern is regular, but not so aggressive that it pollutes the cache on noisy or irregular streams. Uniform stride patterns can benefit from a slightly higher degree, while non-uniform or unstable patterns are better handled with a smaller degree. So the degree of 2 keeps the design conservative at first, and the accuracy-based throttling then decides whether to increase or decrease the aggressiveness by exactly 1 step at a time, with the degree bounded between 1 and 4.

Task 2: I-RRIP and Dynamic I-RRIP

Task 2A: I-RRIP

I-RRIP gives instruction lines a longer expected lifetime in L2. New instruction lines enter at RRPV 1, and data lines enter at RRPV 2. On a hit, either kind of line is promoted to RRPV 0. When choosing a victim, the policy first evicts a data line at the maximum RRPV. If none is available, it evicts an instruction line at the maximum RRPV. If no line has the maximum RRPV, the RRPVs age until a victim is available.

With maximum RRPV 3, instruction insertion at 1 gives an instruction line one extra reuse opportunity compared with the usual RRIP insertion at 2. Data stays at 2 so it remains easier to evict when both classes are old. A hit promotes either class to 0 because recent use is evidence that either line may be reused. At eviction, preferring data at maximum RRPV protects an equally old instruction line when possible. On Trace2, this lowers instruction MPKI from 18.7638 to 15.0475, while data MPKI rises from 8.0330 to 8.9334; the other traces show smaller IPC changes. This is the intended tradeoff: preserve useful code without assuming that data has no value.

The table shows RRIP, I-RRIP, and the selected DIRRIP configuration. Speedup is IPC divided by RRIP IPC for the same trace. L2 load MPKI is also split into instruction and data loads. Complete measurements, including the sensitivity sweep and insertion-position comparisons, are in `data.csv`.

Trace	Policy	IPC	Speedup	L2 load MPKI	L2 instruction MPKI	L2 data MPKI
Trace1	RRIP	0.233047	1.0000	17.1922	10.6175	6.5747
Trace1	I-RRIP	0.233096	1.0002	17.0757	10.0885	6.9872
Trace1	DIRRIP	0.233217	1.0007	17.0432	10.0666	6.9766
Trace2	RRIP	0.250908	1.0000	26.7968	18.7638	8.0330
Trace2	I-RRIP	0.255003	1.0163	23.9808	15.0475	8.9334
Trace2	DIRRIP	0.254821	1.0156	24.0985	15.2079	8.8906
Trace3	RRIP	0.382430	1.0000	3.8467	1.2097	2.6370
Trace3	I-RRIP	0.382608	1.0005	3.7527	1.0845	2.6682
Trace3	DIRRIP	0.382580	1.0004	3.7563	1.0895	2.6668

The results show that I-RRIP lowers instruction-load MPKI on all three traces. The data-load MPKI rises slightly, and IPC gains vary by trace. Thus the policy helps most when the trace benefits from retaining instruction lines.

Task 2B: Dynamic I-RRIP

DIRRIP uses the first 32 L2 sets as RRIP leader sets. They use RRIP for victim selection and insertion, including inserting instruction lines at RRPV 2. The policy counts instruction and data accesses and misses in those sets. This uses a small sample to guide the other sets, though using only the first 32 assumes their traffic represents the cache. For the other sets, it calculates:

```
(instruction accesses + instruction misses) / (all accesses + all misses)
```

If this value is greater than 0.25, it uses I-RRIP; otherwise, it uses RRIP. This follows the assignment's suggested decision rule. The 32-set choice gives the selector a sample of accesses while limiting the cache sets reserved for the RRIP leaders.

I tested thresholds 0.15, 0.20, 0.25, 0.35, and 0.55 with 8, 32, and 64 leader sets. The table gives the geometric-mean speedup across the three traces for each setting.

Threshold	8 leaders	32 leaders	64 leaders
0.15	1.00549x	1.00548x	1.00541x
0.20	1.00544x	1.00555x	1.00547x
0.25	1.00553x	1.00555x	1.00543x
0.35	1.00501x	1.00529x	1.00492x
0.55	1.00012x	1.00021x	0.99996x

The 0.25 threshold and 32 leaders gave the highest average among these settings. The 0.20/32 result was nearly identical, so this small difference should not be treated as a universal optimum. I use 0.25/32 for DIRRIP in the

results table and figures. `data.csv` contains the per-trace measurements for every sweep setting and the additional insertion-at-RRPV-0 comparisons.

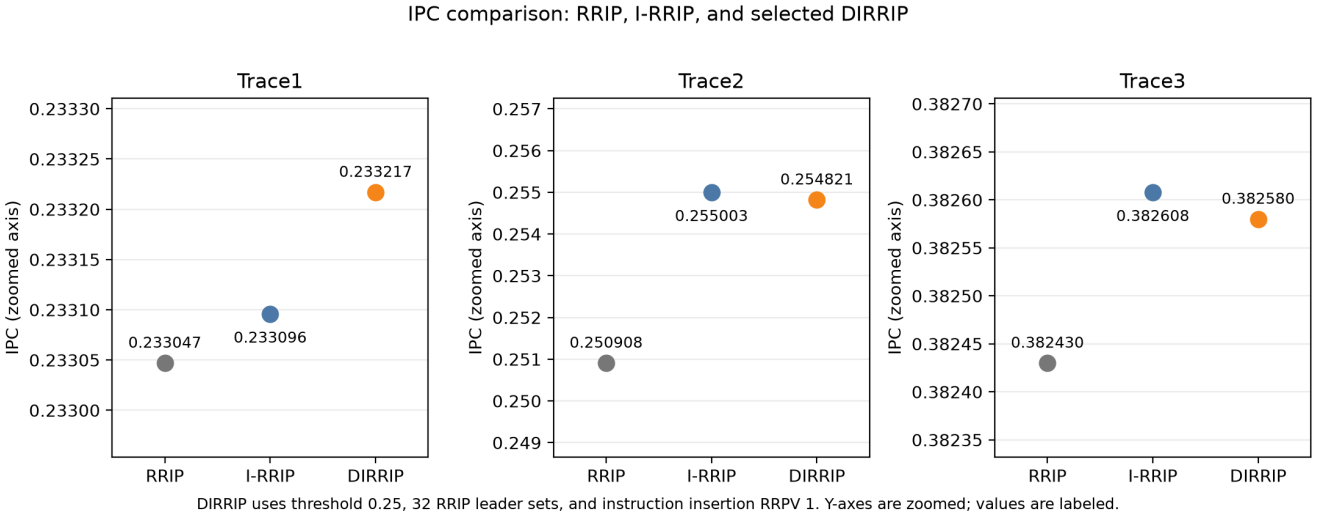
Instruction insertion at RRPV 0 reduces instruction MPKI, but increases data MPKI on Trace1 and Trace2. Insertion at RRPV 1 gives a more balanced result across these traces, so it is retained.

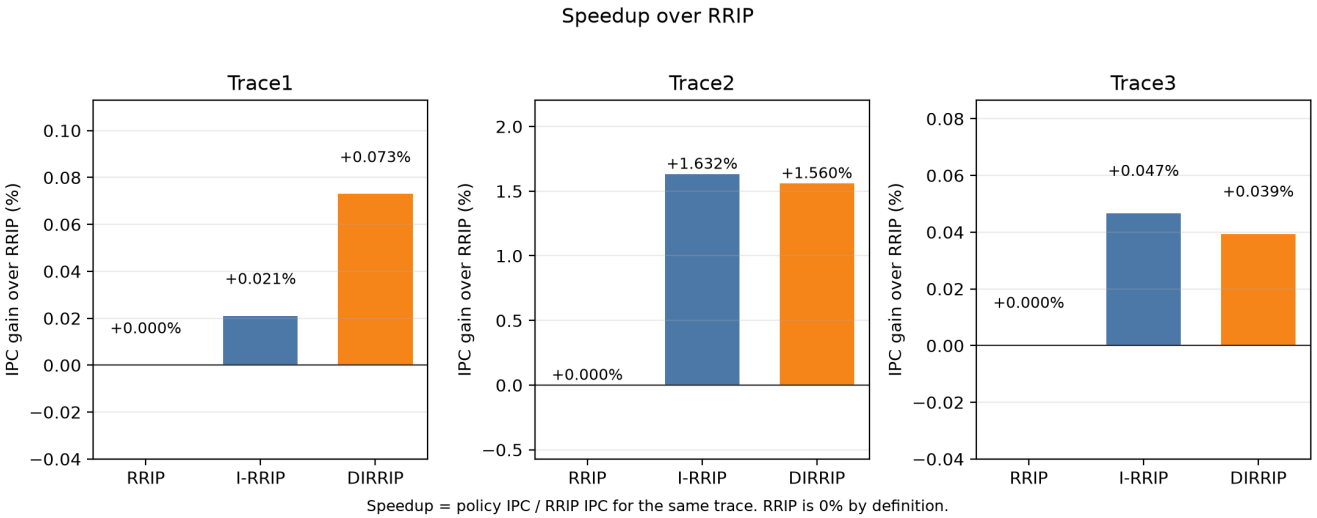
Figures and measurements

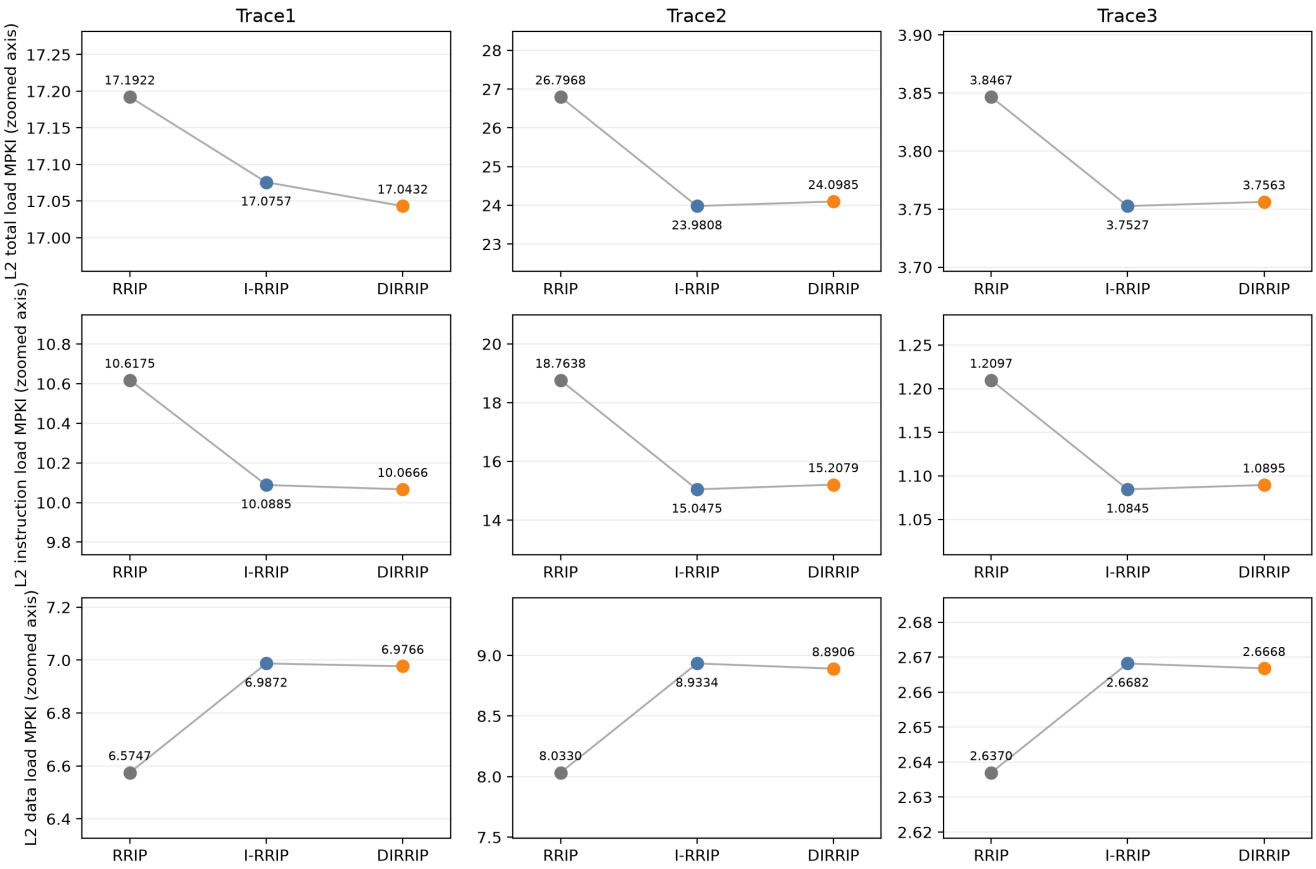
The measurements used a single core, with 25 million warmup instructions and 25 million measured instructions per trace. All policies used the same simulator configuration and `ip_stride` L1D prefetcher. The figures compare RRIP, I-RRIP, and the selected DIRRIP setting (threshold 0.25, 32 leaders). The full sensitivity sweep is reported in the table above and in `data.csv`.

- `ipc.png`: IPC comparison. The y-axis is zoomed for each trace, and each value is labeled.
- `speedup.png`: IPC gain over RRIP, shown as a percentage so small gains are visible.
- `load_mpki.png`: absolute L2 total load, instruction-load, and data-load MPKI. Axes are zoomed by trace and metric, and values are labeled.
- `data.csv`: all plotted and tabulated measurements

The DIRRIP numbers above come from `replacement/dyn_irrip_task2.l2c_repl`. That file is the Task 2 policy, kept unchanged, because `replacement/dyn_irrip.l2c_repl` now carries the prefetch-aware Task 3 version.







Axes are zoomed by trace and metric so small differences are visible; all plotted MPKI values are labeled.

Task 3: Incorporating SPP with Dynamic I-RRIP

The dynamic I-RRIP policy of Task 2 was extended to recognise lines brought into the L2 by the SPP prefetcher. The implementation is in `replacement/dyn_irrip.l2c_repl`. The Task 2 version of the policy is kept as `replacement/dyn_irrip_task2.l2c_repl`.

1) IPC, speedup, prefetch accuracy, coverage and Load MPKI

Trace	Configuration	IPC	Speedup	L1D load MPKI	Prefetch accuracy	Coverage	L2C load MPKI	L2 instruction MPKI	L2 data MPKI
Trace1	Baseline	0.232113	1.0000	10.6788	–	–	17.7251	10.3665	7.3586
Trace1	SPP	0.233918	1.0078	9.6716	62.08%	9.43%	16.9615	10.4284	6.5332
Trace1	Dynamic I-RRIP (no prefetcher)	0.232087	0.9999	10.6784	–	–	17.5826	10.0667	7.5159
Trace1	Task 2	0.233217	1.0048	10.1205	52.83%	5.23%	17.0432	10.0666	6.9766
Trace1	**Task 3**	0.234100	1.0086	9.6712	61.98%	9.44%	16.7171	10.0211	6.6960
Trace2	Baseline	0.249982	1.0000	13.5372	–	–	28.5496	19.8440	8.7056
Trace2	SPP	0.252293	1.0092	13.0892	56.09%	3.31%	28.2278	19.9669	8.2609
Trace2	Dynamic I-RRIP (no prefetcher)	0.254590	1.0184	13.5403	–	–	24.3030	15.3408	8.9622
Trace2	Task 2	0.254821	1.0194	13.4419	37.94%	0.70%	24.0985	15.2079	8.8906
Trace2	**Task 3**	0.257162	1.0287	13.0914	56.24%	3.29%	23.7948	15.3027	8.4921
Trace3	Baseline	0.381868	1.0000	3.4036	–	–	3.9612	1.1265	2.8347
Trace3	SPP	0.384546	1.0070	3.1091	34.17%	8.65%	3.5244	1.1462	2.3782
Trace3	Dynamic I-RRIP (no prefetcher)	0.381716	0.9996	3.4036	–	–	3.9546	1.0771	2.8775
Trace3	Task 2	0.382580	1.0019	3.3176	16.10%	2.53%	3.7563	1.0895	2.6668
Trace3	**Task 3**	0.384431	1.0067	3.1082	34.33%	8.68%	3.4926	1.0747	2.4178

Geometric-mean speedup over the baseline: SPP 1.00801x, dynamic I-RRIP without a prefetcher 1.00594x, **Task 3 1.01462x**.

Task 3 gives the highest IPC overall, and the lowest L2C load MPKI on every trace. Against dynamic I-RRIP it wins on all three traces under both configurations: by 0.867%, 1.010% and 0.711% against the policy alone, and by 0.379%, 0.919% and 0.484% against the Task 2 configuration. Against standalone SPP it wins by 0.078% on Trace1 and 1.930% on Trace2, and loses by 0.030% on Trace3.

Trace2 shows most clearly what the design achieves. Dynamic I-RRIP alone cuts L2 instruction MPKI from 19.844 to 15.341 but pushes data MPKI up from 8.706 to 8.962. SPP alone cuts data MPKI to 8.261 but pushes instruction MPKI up to 19.967. Task 3 reaches 15.303 instruction and 8.492 data, so the instruction benefit survives in full while about 48% of SPP's data-MPKI reduction is kept alongside it.

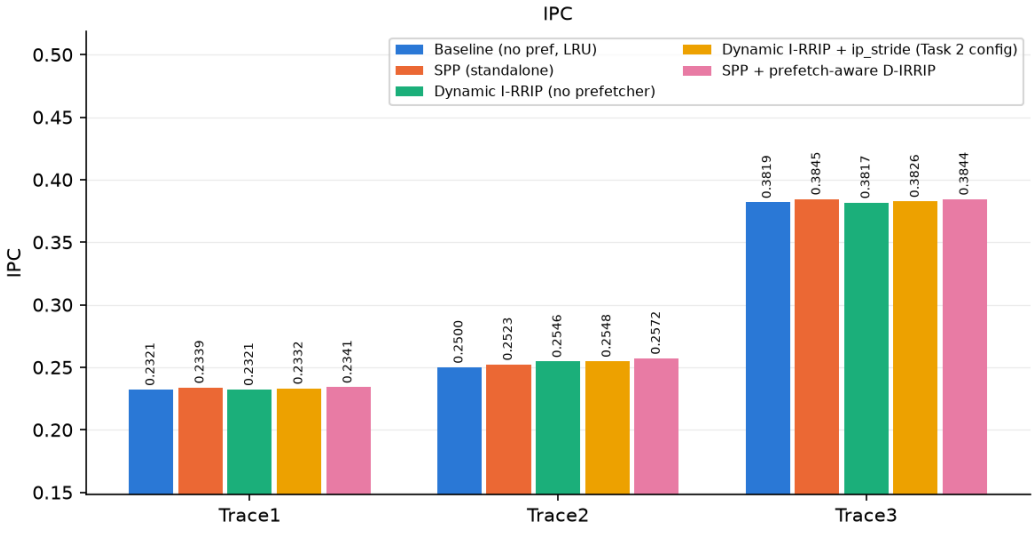


Figure 1: IPC.

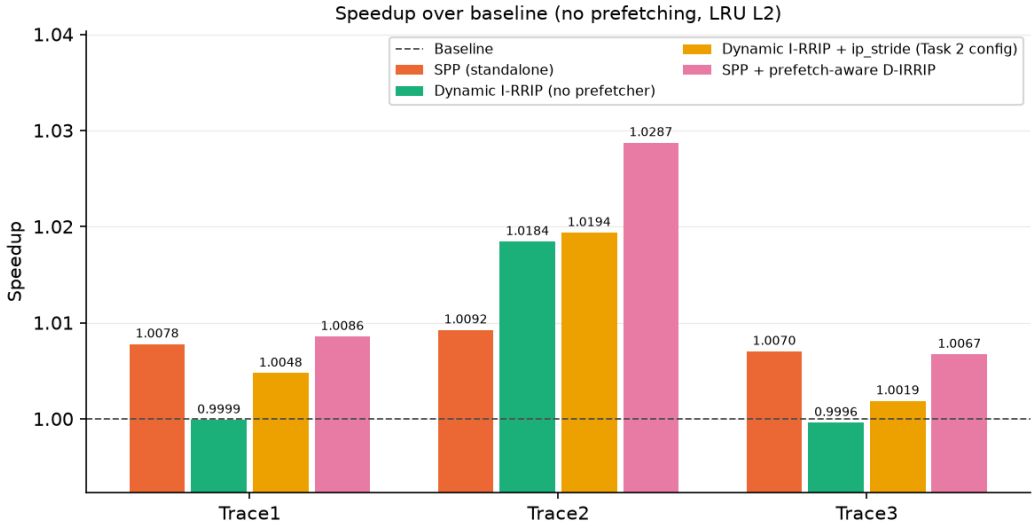


Figure 2: Speedup over the baseline.

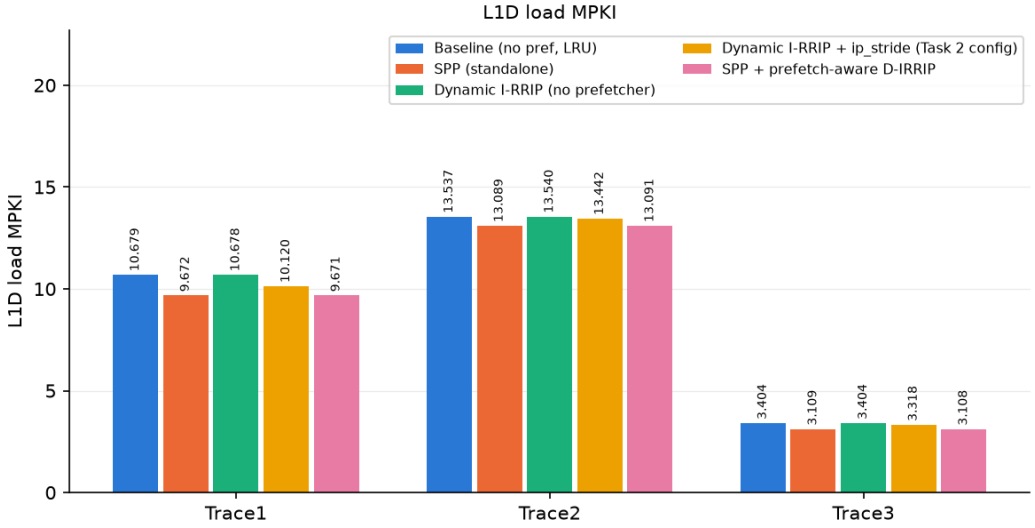


Figure 3: L1D load MPKI.

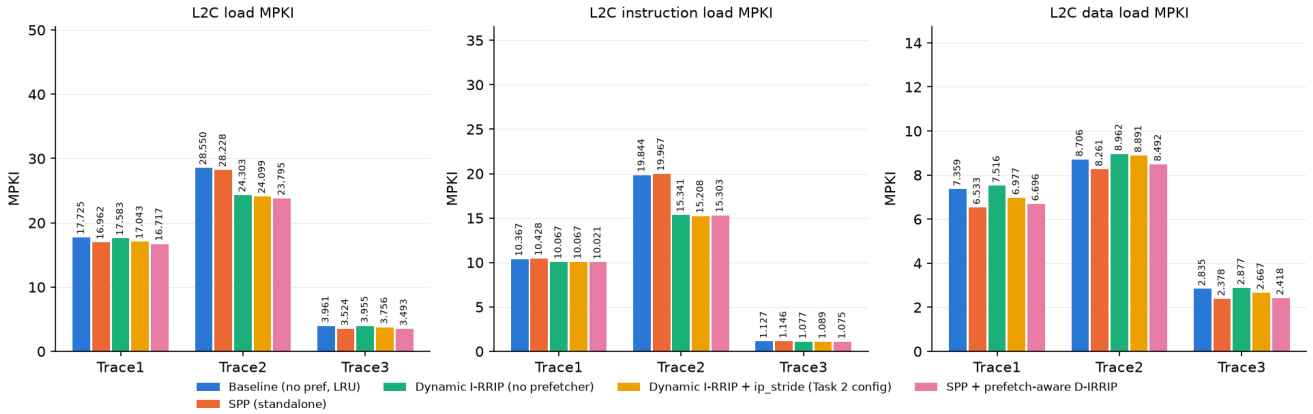


Figure 4: L2C total, instruction and data load MPKI.

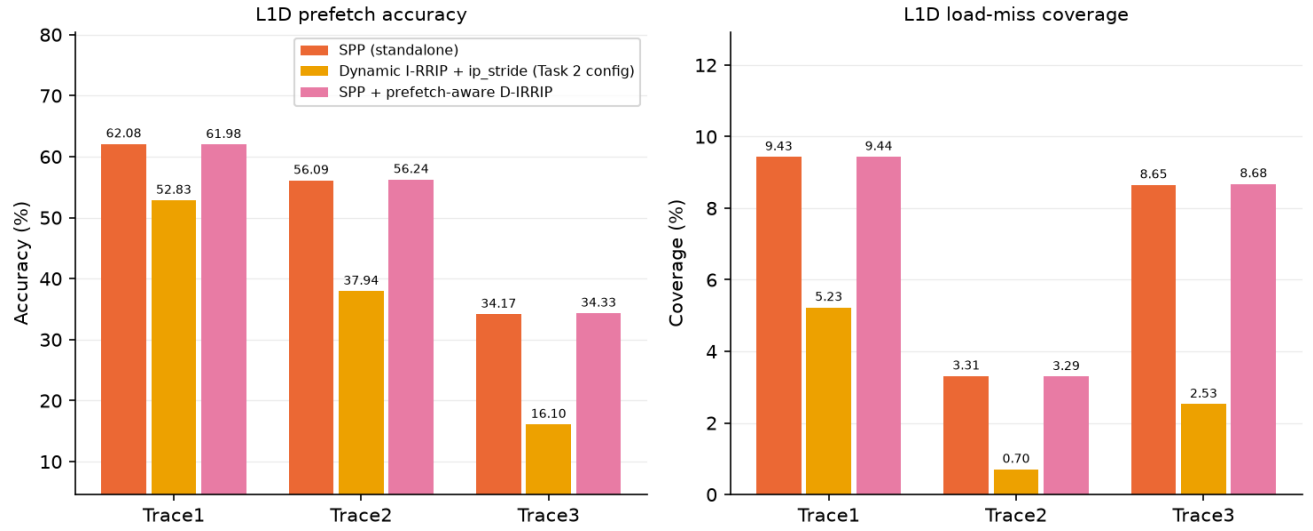


Figure 5: L1D prefetch accuracy and coverage.

2) Rationale for handling the prefetch packet

SPP issues its prefetches with `FILL_L1`, so a prefetched line is installed in the L1D as well as the L2. The program then reads it out of the L1D, and the L2 copy is frequently never touched again from the L2 side. Counters added to the policy measure this over the region of interest: of the prefetch fills into the follower sets, only 10.13% (Trace1), 3.94% (Trace2) and 14.70% (Trace3) are ever used by a later non-prefetch access at the L2. These rates are a property of the Task 3 policy that produced them, so they are quoted as evidence that most prefetched lines do not need a demand-priority slot, not as a measurement of how Task 2 behaved.

The direct evidence that the two mechanisms interfere is in the results table itself. Under the Task 2 policy a prefetched line was inserted at RRPV 2, the same position as a demand data line. Running SPP on its own *raises* L2 instruction MPKI above the baseline on all three traces: 10.3665 to 10.4284 on Trace1, 19.8440 to 19.9669 on Trace2, and 1.1265 to 1.1462 on Trace3. Prefetched data is displacing the instruction lines that I-RRIP exists to protect. Task 3 reverses this on all three traces, reaching 10.0211, 15.3027 and 1.0747, below the baseline in every case.

A `uint8_t is_pf[L2C_SET][L2C_WAY]` array records whether a line was filled by a prefetch and has not yet been used by a non-prefetch access. The test is `type == PREFETCH` exactly, so `PREFETCH_TRANSLATION` and `TRANSLATION_FROM_L1D` are excluded, since those are page-walk traffic rather than SPP lines.

Figure 6 shows the resulting RRPV state machine of one L2 line.

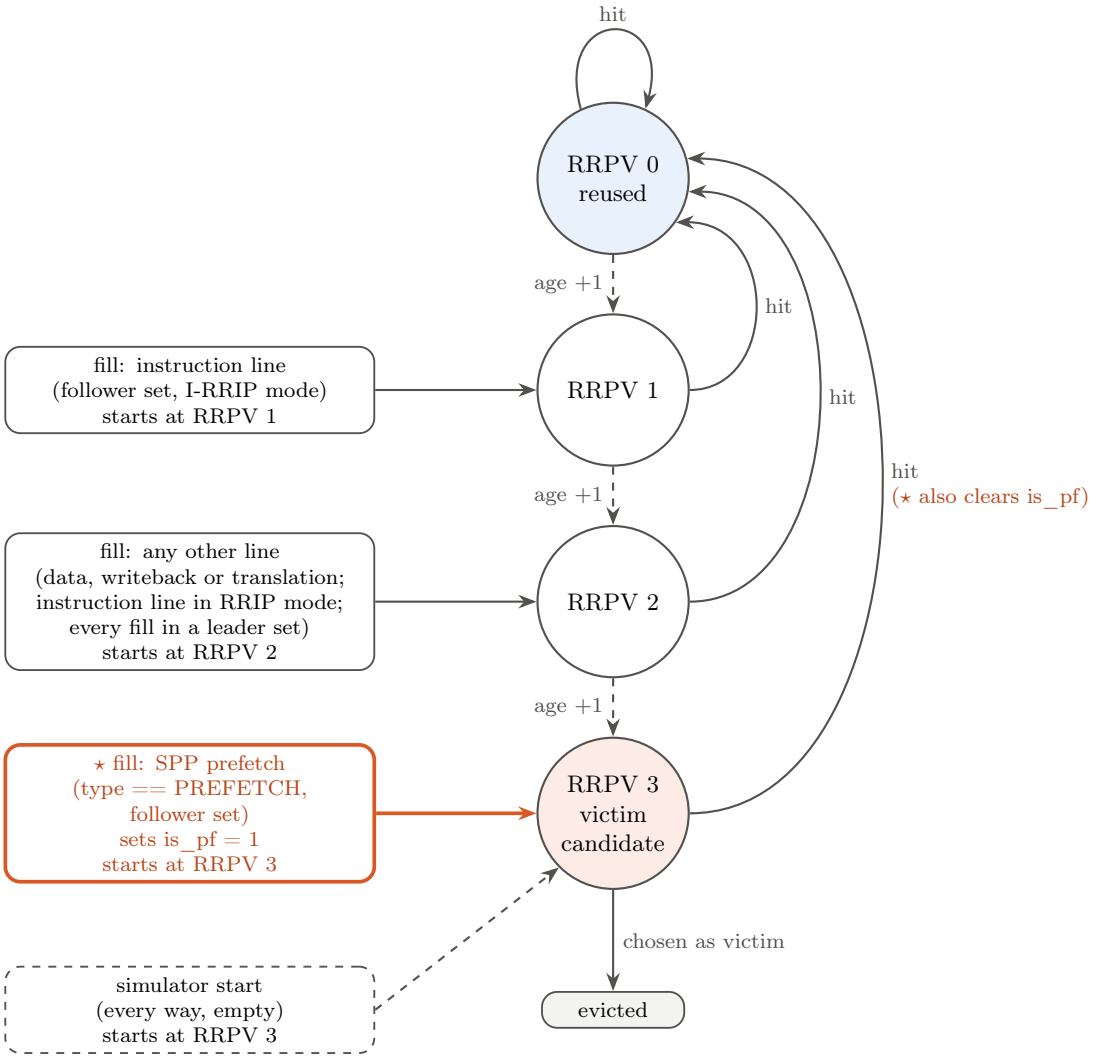


Figure 6: RRPV state machine of one L2 line under the Task 3 policy (`replacement/dyn_irrip.l2c_repl`). Orange with ⋆ marks behaviour changed in Task 3; grey is unchanged Task 2 behaviour. In a follower set a line whose `is_pf` bit is set is always at RRPV 3, which is why only the hit from RRPV 3 clears it.

Insertion – modified. In the follower sets a prefetched line enters at RRPV 3 instead of RRPV 2. It still gets a full chance to be used before the set is next scanned, but it becomes the first candidate when a victim is needed, so it does not go on holding a slot ahead of an instruction line or a demand data line. (The fill itself still evicts a victim, as any fill does; `l2c_find_victim` does not look at the type of the incoming request. What changes is how long the prefetched line then occupies the set.) Together with the eviction preference below, this is what restores the instruction-MPKI reduction.

Promotion – modified. In the follower sets, a hit whose `type` is `PREFETCH` no longer promotes the line to RRPV 0. A prefetch hitting a line the prefetcher itself installed is the prefetcher meeting its own work, not evidence that the program will reuse the line. Under the Task 2 rule a repeated prefetch stream could hold a dead line at RRPV 0 indefinitely, turning the policy into prefetcher-controlled LRU. The rule is applied to every prefetch hit in a follower set, including hits on lines a demand access originally brought in, because in both cases the prefetch itself carries no information about program reuse. A demand hit still promotes to RRPV 0 and clears the prefetch bit, because at that point the prefetch has been proven useful and the line deserves to be treated like any other reused line.

Eviction – modified. The victim scan looks for a line at the maximum RRPV whose prefetch bit is still set before it considers any demand line. This applies in both follower branches, the I-RRIP one and the plain RRIP one, because an unused prefetched line is the cheapest thing to lose whether or not instruction prioritisation is currently active.

Set-dueling counters – modified. Prefetch accesses are no longer counted in the leader-set instruction and data totals. The selector compares `(instruction accesses + instruction misses) / (all accesses + all misses)` against 0.25, and prefetch traffic is entirely on the data side, so counting it dilutes the instruction share and biases the selector toward plain RRIP on exactly the traces that benefit from I-RRIP. This changes what the selector observes, not how the leader sets themselves replace lines.

Leader sets – left as plain RRIP. The three policy changes above are confined to the follower sets. Sets 0 to 31 keep plain RRIP insertion, promotion and eviction exactly as in Task 2, so the duel retains an unmodified control group.

What was left alone. The instruction and demand-data insertion positions (RRPV 1 under I-RRIP, RRPV 2 otherwise), the demand-hit promotion to RRPV 0, the ageing loop, the 32 leader sets and the 0.25 threshold are all unchanged from Task 2. The problem being fixed is specific to prefetched lines, so the rest of the policy had no reason to move, and holding it fixed makes the comparison against Task 2 a clean measurement of the prefetch handling alone.